

TEL102 - Control 3

Semestre 2, 2025

Fecha: 28/10/2025, Duración total: 1h 10min, Duración Parte A: Máximo 35 min.

Instrucciones generales - Parte A

- **Parte A:** resolución **en papel**, sin uso de medios externos.
 - Se debe diseñar un conjunto de clases en **C++**, utilizando únicamente los conceptos vistos en clases.
 - El diseño debe ser compatible con el código en **C** entregado y con el **ejemplo de uso en C++**.
 - Una vez entregada esta parte, podrá comenzar la Parte B, con un tiempo máximo de 35 minutos para esta primera sección.
-

Entregables

- **Parte A (papel):** hoja con el diseño de las clases **Mascota**, **Perro** y **Gato**.
-

Parte A: Diseño de clases (35 min)

El **Refugio FuturPaws** mantiene un registro de mascotas que llegan al centro de adopción. Cada mascota tiene un **nombre**, una **edad** y un **tipo** (Perro o Gato). El sistema actual está implementado en **C** y permite registrar, listar y calcular el promedio de edad. En este control, el objetivo es **rediseñar el programa en C++** aplicando **Programación Orientada a Objetos**, donde los distintos tipos de mascotas se representen mediante **herencia**.

A partir del código en **C** del refugio de mascotas (**refugio.h**, **refugio.c**, **main.c**), y del **ejemplo de uso en C++** provisto, diseñe las clases:

```
Mascota    // Clase base
Perro      // Hereda de Mascota
Gato       // Hereda de Mascota
```

El código en C utiliza el campo **tipo** para diferenciar entre perros y gatos. En la versión orientada a objetos, esta distinción debe lograrse **mediante herencia**, definiendo las clases **Perro** y **Gato** como derivadas de **Mascota**, con sus propios **atributos** y **métodos**.

Ejemplo de uso C++ (referencial):

```
std::vector<Mascota*> refugio;  
// Entradas de cada constructor variarán según su propio diseño  
refugio.push_back(new Perro(...));  
refugio.push_back(new Gato(...));  
for (int i = 0; i < (int)refugio.size(); i++) {  
    refugio[i]->imprimir();  
}
```

Puntajes Parte A (50 pts)

- Estructura base y herencia correcta: 15 pts
- Atributos coherentes (base + específicos): 10 pts
- Métodos principales y firmas compatibles: 15 pts
- Claridad del diseño y legibilidad: 10 pts

Códigos

refugio.h

```
struct Mascota {
    int id;
    char nombre[32];
    char tipo[16];    /* "Perro" o "Gato" */
    int edad;
};

struct Refugio {
    struct Mascota* lista;
    int cantidad;
    int capacidad;
};

/* Reserva memoria para 'capacidad' mascotas. Retorna 1 si OK, 0 si falla. */
int refugio_inicializar(struct Refugio* r, int capacidad);

/* Libera memoria y deja el refugio en estado vacío. */
void refugio_liberar(struct Refugio* r);

/* Agrega una mascota si hay espacio. Retorna 1 si OK, 0 si está lleno. */
int refugio_agregar(struct Refugio* r, struct Mascota m);

/* Retorna el promedio de edad; si no hay mascotas, retorna 0.0. */
double refugio_promedio_edad(struct Refugio* r);
```

refugio.c

```
#include "refugio.h"
#include <stdlib.h>

int refugio_inicializar(struct Refugio* r, int capacidad) {
    r->lista = (struct Mascota*)malloc(sizeof(struct Mascota) * capacidad);
    if (r->lista == NULL) {
        r->cantidad = 0;
        r->capacidad = 0;
        return 0;
    }
    r->cantidad = 0;
    r->capacidad = capacidad;
    return 1;
}

void refugio_liberar(struct Refugio* r) {
    free(r->lista);
    r->lista = NULL;
    r->cantidad = 0;
    r->capacidad = 0;
}

int refugio_agregar(struct Refugio* r, struct Mascota m) {
    if (r->cantidad >= r->capacidad) return 0;
    r->lista[r->cantidad] = m;
    r->cantidad = r->cantidad + 1;
    return 1;
}

double refugio_promedio_edad(struct Refugio* r) {
    if (r->cantidad == 0) return 0.0;
    int i = 0;
    long suma = 0;
    while (i < r->cantidad) {
        suma = suma + r->lista[i].edad;
        i = i + 1;
    }
    return (double)suma / (double)r->cantidad;
}
```

main.c

```
#include "refugio.h"
#include <stdio.h>

int main() {
    struct Refugio refugio;

    if (!refugio_inicializar(&refugio, 3)) {
        printf("Error al reservar memoria.\n");
        return 1;
    }

    for (int i = 0; i < refugio.capacidad; i++) {
        struct Mascota nueva;
        printf("\n--- Registro de Mascota %d ---\n", i + 1);
        printf("ID: ");
        scanf("%d", &nueva.id);
        printf("Nombre: ");
        scanf("%31s", nueva.nombre);
        printf("Tipo (Perro o Gato): ");
        scanf("%15s", nueva.tipo);
        printf("Edad: ");
        scanf("%d", &nueva.edad);

        if (!refugio_agregar(&refugio, nueva)) {
            printf("No hay espacio disponible.\n");
            break;
        }
    }

    printf("\nMascotas registradas: %d\n", refugio.cantidad);
    for (int i = 0; i < refugio.cantidad; i++) {
        printf("%d) %s, %s, %d años\n",
            refugio.lista[i].id,
            refugio.lista[i].nombre,
            refugio.lista[i].tipo,
            refugio.lista[i].edad);
    }

    double promedio = refugio_promedio_edad(&refugio);
    printf("\nPromedio de edad: %.2f años\n", promedio);

    refugio_liberar(&refugio);
    return 0;
}
```